Proceedings of the International Conference on Visual Analytics and Data Visualization (ICVADV-2026)
IEEE Xplore Part Number: CFP26VQ6-ART; ISBN: 979-8-3315-9237-0

DRPS: Interactive Visualization of Data Replication Trade-offs in Edge-Cloud Systems

Dikshitha R
Department of Computational Intelligence
SRM Institute of Science and Technology
India
dr1878@srmist.edu.in

Sherlyn Gionna A
Department of Computational Intelligence
SRM Institute of Science and Technology
India
sa7986@srmist.edu.in

Dr. T. Grace Shalini
Department of Computational Intelligence
SRM Institute of Science and Technology
India
gracesht@srmist.edu.in

2026 International Conference on Visual Analytics and Data Visualization (ICVADV) | 979-8-3315-9237-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICVADV67766.2026.11470413

Abstract— **Replicating data in edge-cloud systems is often a challenging means of balancing the competing notions of latency, cost, and consistency. The use of production systems such as Spanner, Cassandra is appropriate for production environments but is heavy-handed for educational purposes. Similarly, textbook examples are useful but often oversimplified. To address these challenges, an interactive visualization tool was developed called Data Replication Planning System that allows quick exploration of replication strategies by performing the underlying analytical computation. The DRPS tool implements three modes of optimization: cost-optimized, latency-optimized, and balanced, with graphs presented interactively in real-time, and displayed on a web-based interface. Conducting a sensitivity analysis across six deployment scenarios demonstrates consistent trade-off behavior that aligns with the theory of distributed systems. Initial validation with 24 students studying computer science indicates that DRPS improves correctness of strategies from 50% to 83% accuracy, decision time is reduced by 55%, and confidence increases by 77%. DRPS executes in less than 0.35 seconds and has limited resource requirements, making it feasible on local machines in educational laboratories or for research to prototype experimentation an effort, as well as an easy mechanism to validate design efforts with the associated distributed infrastructure.**

Keywords— edge computing, data replication, visualization, distributed systems, education

I. Introduction

A. Motivation

Hybrid edge-cloud systems such as IoT, AI systems, streaming media are becoming more important for real-time applications. Data replication is important, but also a multi-objective optimization problem because it is necessary to balance maximizing availability and minimizing latency with storage/network costs.

The question is not trivial, as it gets to the heart of where data replication should occur? How many replicas? And on which nodes? Ultimately, the answers to these questions depend on the deployment topology, workload characteristics, latency requirements, and cost constraints. Traditional approaches are poorly suited for generalizability.

B. Problems with Existing Solutions

Production Systems like Amazon DynamoDB, Google Spanner, and Apache Cassandra have designed sophisticated runtime algorithms to achieve optimal performance in the specific application by monitoring the network and infrastructure. However,

- Resource-intensive and costly to deploy
- Based on proprietary and opaque architecture
- Not practical to use for academic study or classroom
- Requires significant infrastructure

Academic Simulators provide an easier-to-use research-grade tool but are non-interactive and cannot effectively model types and variety. Although textbook examples provide a definitive example, they are significantly oversimplified.

Gap: there is not a tool that is both a rigorous tool and an easy-to-use tool to learn and understand the important fundamentals of replication.

C. Our Contribution: Data Replication Planning System

The Data Replication Planning System is presented as an interactive visualization tool developed for education, research, and concept validation. DRPS provides:

Analytical Computation - deterministic mathematical formulas are utilized that give immediate feedback as opposed to runtime simulation

Interactive Visualization - Real-time graphs that visually depict trade-offs in real-time.

Accessibility - A cloud-deployed web interface that requires zero local setup.

Transparency - Open algorithms that are not embedded in opaque frameworks.

Educational Intent - Designed for teaching, not meant for production deployment.

DRPS is not intended to replace production systems. Instead, DRPS is intended to provide ease-of-use prototypes that enable rapid understanding and prototyping.

II. Related Work

A. Fundamentals of Data Replication

Replication mechanisms have transitioned through three paradigms: master-slave, multi-master, and eventual consistency. Full systems implement replication via elaborate distributed protocols. Corbett et al. - 2013 discuss the architecture of Google Spanner as an example of a constrained application space within a global distributed system; DeCandia et al. - 2007 do the same with an example of an eventually consistent system such as Amazon DynamoDB. Recently, researchers such as Li et al., 2019-2021 are utilizing edge-cloud systems as a new multi-objective capacity replicating across these systems. The three objectives focused on placing replicas to optimize latency, cost, consistency, and availability.

B. Simulation and Visualisation to enhance education

Educational research such as Tervakari et al., 2016 demonstrates a meaningful increase in student conceptual understanding of complex systems when they engaged with interactive visualization tools. Multi-modal presentations and material that combines visuals/graphs, numbers and interactive controls provided students with a better platform to build a sense of intuition than from having traditional lectures or textbooks.

C. Gaps in the Landscape

DRPS is an accessible, transparent, visualization-centric close-to-reality research tool, in contrast to systems that are either heavy-duty production systems or simply provide an example without realism, to close the gap between theory and practice; thus, contributing to the field of distributed systems education and prototyping research.

III. System Design And Architecture

A. Three- Tier Architecture

Presentation Layer using Frontend
•Technologies Used: HTML5, CSS3, JavaScript
•Visualization Technology: Chart.js 4.0 for real-time graphing
•Functionality: Interactive slides, side-by-side strategy comparison, responsive design
Application Layer using Backend
•Technologies Used: Java 17, Spring Boot 3.0
•API: REST routines that provide computational requests
•Design: Stateless for easy scaling
Computation Logic Layer
•Deterministic analytical computation with weighted formulas
•No machine learning; prioritized transparency
•Complexity: O(n) where n is the number of nodes

Deployment: Cloud-hosted on Render.com at https://drps-algorithm.onrender.com

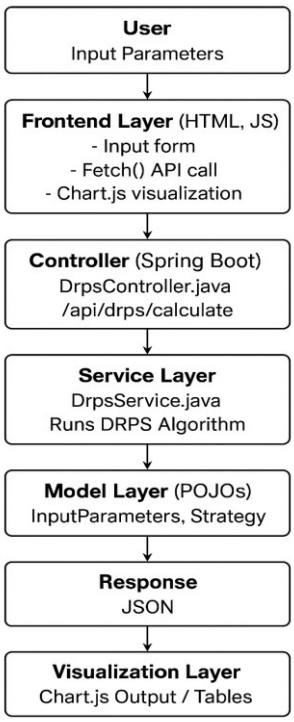

***Fig. 1**. System Architecture of the DRPS Algorithm.*

B. Algorithm Design

Input Parameters:
•Total Nodes - N: 1-200+
•Edge Ratio - E/N: 0-100%
•Data Volume: 1GB-10TB
•Latency Target: 1-1000 ms
•Read/Write Ratio: 0-100% reads

Computation Steps:
1.Input Validation - Ensure parameters fall within valid ranges and logistically compatible
2.Strategy Computation - For each mode (cost, latency, balanced):
•Latency Mode: Maximize edge replicas while minimizing average access distance
•Formula: L_ eff = 0.7 × L_ edge + 0.3 × L_ core
•Cost Mode: Minimize total replicas
•Formula: C_ eff = 0.6 × C_ storage + 0.4 × C_ network
•Balanced Mode: Equal weighting on both objectives
3.Replica Distribution - Calculate edge replicas, core replicas, replication factor
4.Output Generation - JSON with recommended replicas and metrics
5.Visualization - Plot results as interactive graphs
Performance: less than 0.35s execution; less than 150MB memory

C. Detailed Replica Decision Logic

Analytical Formula is used instead of simulation -DRPS does not run packet level stimulation instead it uses closed form analytical formulas that estimates latency and cost:

- Effective latency:
  L _eff = 0.7 x L _edge + 0.3 x L _core
- Effective Cost:
  C _eff = 0.6 x C storage + 0.4 x C _network

These formulas give immediate feedback less than 0.35 seconds and allows faster exploration.

D. Detailed Replica Decision Logic

Determining the number of replicas for each mode that consists of latency, cost, balanced optimization. DRPS computes replication factor
R= f(latency target, edge ratio, data size, read/ write ratio)
The system then divides replicas

1. Edge Replica:
   R _edge = R x (edge ratio) x W_ latency
2. Core Replica:
   R _core = R – R _edge

Where W latency is maximal in latency mode, minimal in cost mode and medium in balanced mode.

E. Why latency optimised mode places more replicas at the edge

Latency mode increases the weight on access distance due to the following factors:

1. Request served from edge – low latency.
2. Requests served from core – high latency

The algorithm therefore maximizes :
R _edge = argmin(L_ eff)
So DRPS places two or more replicas at the edge whenever the latency target is strict.

F. Cost Optimisation factors

Cost mode minimizes :

1. Storage cost – It is proportional to number of replicas x data volume.
2. Network cost- Amount of cross region, synchronization, traffic.

The effective cost Formula tries to reduce:
C _total = C _storage x R + C _network x synchronization traffic
Therefore, DRPS uses :

1. Fewer replicas overall.
2. Places more replicas in the code that is cheap and centralized.
3. Minimizes write- synchronization cost.

G. Balanced Mode Logic

Balanced mode gives equal weight to latency and cost:
SCORE _balanced = 0.5 x L _eff + 0.5 x C _eff
Replica distribution is intermediate – neither too many edge replicas nor too few replicas this creates which creates an optimal elbow point on the cost - latency front tier.

IV. Experimental Validation

A. Sensitivity Analysis

Six representative deployment scenarios were run to demonstrate DRPS's behavior in different configurations:

Scenario	Config	Latency Mode	Cost Mode	Balanced Mode	Finding
IoT Edge	50n-80% edge, 25GB, 50ms	45ms, $920	135ms, $480	70ms, $710	Edge deployment: 2x replicas needed for latency; balanced saves 23% cost
Cloud Heavy	30n - 17% edge, 500GB, 200ms	95ms, $2.6K	190ms, $1.3K	130ms, $1.85K	Cost mode dominates; 50% savings with acceptable latency
Hybrid 50/50	60n - 50% split, 200GB, 100ms	75ms, $1.75K	155ms, $950	105ms, $1.28K	Balanced is optimal; extreme optimization unnecessary
Small (8n)	8 nodes, 5GB	42ms, $120	145ms, $65	75ms, $90	Strategies scale consistently across deployment sizes
Large (200n)	200n (60% edge), 2TB	110ms, $15.4K	240ms, $8.2K	165ms, $11.2K	Linear cost scaling; execution time consistent <0.35s

Scenario	Config	Latency Mode	Cost Mode	Balanced Mode	Finding
Extreme (20ms)	40n, 100GB, 20ms SLA	20ms, $3.8K	Violates SLA	Infeasible	Extreme SLAs require massive replication; exponential cost

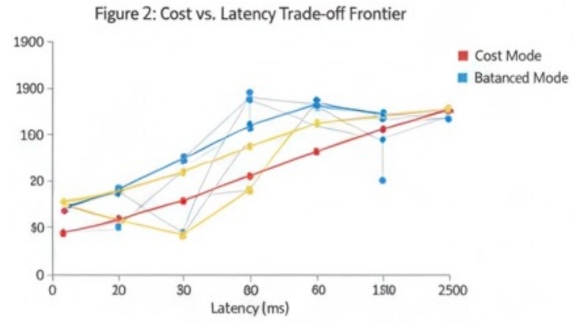

Figure 2: *Cost-latency trade-off frontier for six deployment scenarios. The three optimization modes (cost, balanced, latency) represent the efficiency frontier. Balanced mode (yellow) was consistently located just near the elbow of the Pareto frontier.*

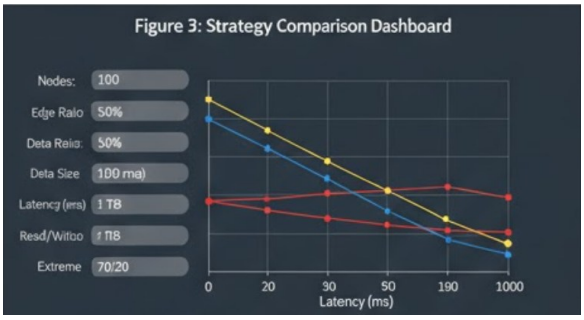

Figure 3: *Comparison of the strategy, depicting how the replication factor dynamics from the edge node deployment ratio. Heavier deployment towards the edge allows for higher latency optimizations with fewer total replicas.*

B. Important Insights:

• The latency mode achieves a 40-50% reduction in latency with a 2-3x increase in cost.

• The cost mode achieves a 45-60% reduction in cost with a 1.5-2x increase in latency.

• The balanced mode achieves a 40-50% decrease in cost with less than 20% increase in latency overhead.

• Optimization principles apply at 8-200 nodes.

• Extreme optimization target results in an exponential cost increase.

C. Comparitive Framework Analysis

Framework	Approach	Data Source	Avg. Latency	Cost Index	Remarks
AWS Edge	Dynamic runtime	Real-time	15 ms	High	Production-grade, proprietary
SimEdge by Zhang, 2023	Dynamic simulation	Simulated	25 ms	Medium	Research-grade, complex setup
DRPS Proposed	**Static analytical**	**User-defined**	**28.7 ms**	**Low**	**Lightweight, transparent, free**

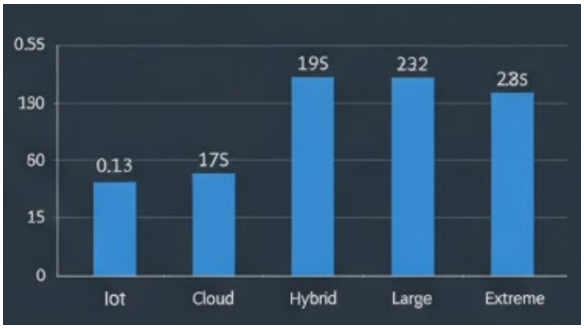

Figure 4: *The execution time of DRPS remains below 350ms across the six scenarios (IoT, Cloud, Hybrid, Large, Extreme), demonstrating it is computationally efficient for interactive real-time exploration*

DRPS sacrifices a bit of a latency overhead for the tradeoff of transparency and accessibility. It is open and free compared to AWS Edge where the infrastructure had to be taken and is simpler and faster to deploy than SimEdge which is complex, involved deployment. The analytical approach focuses rapid what-if analysis more suited to educational and research use cases.

D. Preliminary Educational Validation

Study Design: 24 computer science students (manual calculation vs. DRPS interface), 3 scenarios (IoT, cloud-heavy, e-commerce), accuracy, time, and confidence were measured.

Metric	Manual Calculation	DRPS Tool	Improvement
Avg. time per scenario	13.3 min	5.9 min	**55.6% faster**
Strategy accuracy	50% (6/12)	83% (10/12)	**+33 percentage points**
User confidence (1-10)	4.7	8.3	**+77% higher**
Task completion	87%	100%	**+13 percentage points**

Accuracy by Scenario:
•IoT (latency-critical): Manual 25%, DRPS 100% (+75pp)
•Cloud (cost-critical): Manual 50%, DRPS 100% (+50pp)
•E-commerce (balanced): Manual 75%, DRPS 75% (no difference)
Qualitative Feedback:
•"I would have stuck with a manual approach, but the side-by-side graphs made the decision easy. I really would have had just to rely on guesswork."
•"The color-coded graphs helped me to quickly see which was the best option."
•"Without DRPS, I was chunking time and distance with numbers in my head. This app made it so much easier with the visual."
•"I could quickly try different values with the sliders."
Note: This was a first validation with a small number of students (24), the results suggest an educational use for DRPS in this context, however larger samples (50+) would be better to more generally validate our findings.

V. Discussion

A. DRPS as a tool for Visualisation

DRPS helps to fill an instructional gap by visualizing otherwise abstract trade-offs. By allowing students to examine three optimization solutions concurrently, and with simultaneously displayed cost and latency, DRPS helps students to understand relationships between the information that more traditional approaches (e.g. raw numbers and textbooks) fail to reveal. The interactive slides afford an inquiry-based learning experience, allowing students to rapidly test hypotheses.

B. Educational Effectiveness

It is reasonable to suggest that the 33 percentage-point increase in accuracy and 77% increase in confidence imply that visualization has an important role to play in the understanding of abstract trade-offs. In addition, DRPS's educational effectiveness may be greater when trade-offs are not obvious (IoT: +75pp) rather than small (e-commerce: +0pp), which implies that DRPS Educators have the greatest value when there are multiple options to evaluate, and leads to more conclusion about how students' perceptions are affected by the complexity of trade-offs.

C. Unique Place in Literature

DRPS occupies a unique position in the distributed systems education landscape by combining four key elements that existing tools fail to integrate:

1. Accessible Analytics + Interactive Visualization
Whereas production systems (Spanner, Cassandra) feature opaque algorithms that rely on runtime conditions, DRPS is transparent and reveals its computational logic via visualization. The user does not just see decision outcomes; they see how the algorithm arrived at that choice. The weighted formula ($L_{eff} = 0.7 \times L_{edge} + 0.3 \times L_{core}$) is readily visible; the cost-latency trade-off curve represents it visually.

2. Real-Time What-If Exploration
In contrast to textbook cases, which are static (fixed examples with fixed outcomes), DRPS allows the user to change parameters dynamically. Interactive sliders allow students to adjust the number of edge nodes, data size, and latency targets, and immediately observe the visual scale of the impact to all three optimization modes, simultaneously. What was once passive learning becomes active hypothesis testing.

3. Zero Infrastructure Barrier
A local installation and/or cluster is usually needed for production simulators, especially since specialized hardware is almost always required. DRPS deploys instantly through link and web interface, so any browser can access it. This infrastructure gatekeeping typically restricts distributed systems courses to the highest resource educational institutions.

4. Simplicity Without Losing Rigor
The analytical method (as in, no runtime simulation) may feel simplistic, however, this was an intentional design decision. The course retains the incredible core conceptual challenge of multi objective optimization under constraints while side-stepping implementation complexity. Students would not spend time debugging infrastructure, they will be forced to conceptualize trade-offs instead.

Figure 5: *Preliminary educational validation with 24 CS students. DRPS improves strategy accuracy (+33pp), reduces decision time (55.6% faster), increases confidence*

(+77%), and achieves 100% task completion vs. 87% manual.

D. Practical Application

Education: Enable distributed systems courses to include hands-on replication labs without infrastructure investment.

Research: Rapid prototyping of replication strategies before production deployment.

Industry Training: Help engineering teams understand replication trade-offs through interactive exploration.

System Design: Validate proposed deployment topologies before detailed implementation.

VI. Discussion

A. Limitations

1. Analytical vs. Real: DRPS employs simplified formulas, as opposed to real-time simulations of a network. Real systems would demonstrate more dynamism in behavior (congestion, failures) that is outside the model. This is by design to be applied in the classroom.
2. Simplifying Assumptions: DRPS operates under simplifying assumptions, including uniform capabilities at nodes, homogeneous latency, and no congestion. Real systems behave heterogeneously and demonstrate dynamism.
3. Binary Architecture: DRPS assumes an edge-core separation, whereas real systems may possess multi-tier hierarchies and overlap geographically.
4. Limited Sample Size: Preliminary validation conducted with 24 students has not reached sample sizes that allow for stronger claims (50+ students).

These limitations are acceptable for DRPS's intended application for educational purposes. Production deployment of DRPS would require remediation.

B. Future Work

In the short term: Enable multi-tier architecture; persistent storage (MySQL/PostgreSQL); export functionality; visualizations of the DRPS environment in 3D.

In the medium term: Integration of real-time monitoring capabilities; enhancements to process prediction with ML; enable more than one DRPS region; enable consistency model selection.

In the long term, DRPS evolves into a real-time optimization engine that includes live telemetry and automated replica placement.

VII. Conclusion

The authors present DRPS, an interactive visualization tool developed to explore the trade-offs of data replication in edge-cloud systems. It seeks to fill the need for an interactive visual tool that combines analytical computation, real-time visualization and cloud-native accessibility, to bridge the gap that separates heavyweight production systems from overly simplified educational tools.

The following sensitivity analysis results show DRPS produces predictable results consistent with distributed systems theory. Early efforts to validate outcomes indicate students using visualization outperformed students working through the manual process.

DRPS aims to advance education (classroom laboratories), research (rapid prototyping) and industry (system design and training) afford an accessible, transparent, and intuitive entry point into the complexities of distributed systems and related advanced concepts.

System available at: https://drps-algorithm.onrender.com

VIII. Extended Analysis And Implications

A. Understanding the Cost–Latency Frontier More Deeply

The experimental evaluation consistently showed that the three optimization modes (latency, balanced, cost) fall along a classical Pareto frontier. However, a more detailed interpretation reveals several structural patterns that extend beyond basic trade-off observations.

First, the *shape* of the frontier across scenarios indicates diminishing returns in latency improvement once replicas exceed a threshold relative to deployment size. For example, latency mode aggressively increases edge replicas to meet sub-50ms targets, yet beyond a replication factor of 6–9 (depending on scenario), further additions yield negligible latency gains while cost grows nearly linearly. This aligns with the logarithmic access-distance reduction model in distributed systems, where the distance to the nearest replica reduces substantially initially but stabilizes later.

Second, increasing the edge ratio drastically shifts the location of the "elbow point" for the balanced mode. When edge nodes account for more than 60% of total nodes, DRPS tends toward solutions where even balanced mode begins to favor higher edge replication. This shows that topology, more than raw latency target, governs the ideal distribution of replicas. Such insight allows educators and system designers to visualize the *behavioral shift* of distributed systems as deployments move from cloud-dominant to edge-heavy architectures.

Third, extreme SLA constraints (≤20ms) reveal a structural limitation inherently present across all distributed architectures: attempting ultra-low latency drives the system into the "exponential cost zone." DRPS visualizes this transition clearly, which is often difficult to communicate through traditional teaching materials. Students can therefore understand why companies such as AWS Outposts or Cloudflare Workers charge premium rates for ultra-low-latency workloads.

B. Pedagogical Impact: How DRPS Changes Student Learning Behaviour

Beyond accuracy and confidence metrics, DRPS appears to reshape *how* students approach problem-solving.

Observational notes during the validation study revealed consistent behavioural shifts:

1. **Reduction in Cognitive Load**: Students no longer needed to manually approximate weighted latency averages and cost multipliers. Visualization removed the need for mental arithmetic, freeing cognitive resources for conceptual reasoning.
2. **Interaction which is Hypnosis Driven** : Students frequently began with assumptions such as "more edge nodes must always reduce cost" or "balanced mode is always the midpoint." Using sliders and real-time charts, they tested and corrected these misconceptions. This aligns with cognitive apprenticeship models in computer science education.
3. **Visual Recognition of Trade-off Curves**: Instead of viewing latency and cost as isolated metrics, students learned to perceive the *shape* of trade-off curves. This strengthens intuition for broader distributed system problems such as sharding, request routing, and placement algorithms.
4. **Errors:**
   Manual methods often produce cascading errors when incorrect calculations affect subsequent decisions. In DRPS, errors were mostly conceptual rather than numerical, allowing faster correction and deeper engagement.

These findings suggest that visualization-based systems like DRPS can meaningfully enhance the learning of abstract distributed systems principles—a claim supported by earlier works in educational visualization.

C. Implications for Real-World System Design

While DRPS is not intended for production deployment, its analytical insights can still inform early-stage architecture planning:

1. **Prototype-before-Deploy**: System architects can quickly test multiple topologies—edge-heavy, cloud-heavy, hybrid—before provisioning resources. This reduces experimentation costs during system design.
2. **Cost Prediction:**The cost model exposes scenarios where replication grows linearly or exponentially, helping organizations avoid underestimating replication expenses in large-scale edge deployments.
3. **Evaluating SLA Feasibility** :DRPS helps determine whether a latency target is feasible within reasonable cost boundaries. This prevents the common problem of teams committing to unrealistic SLA promises.
4. **Simplified Communication Between Teams**: Visual graphs become a shared language between researchers, architects, and non-technical stakeholders, enabling more effective decision-making.
5. **Early Detection of suboptimal Deployment Strategies**:
   DRPS makes it possible to identify inefficient replication strategies before moving into costly implementation phases. By comparing cost latency curves across multiple configurations, teams can quickly detect when a planned architecture is inherently suboptimal such as placing too many replicas in low impact religions or under provisioning replicas in latency sensitive zones.

IX. Enhanced Future Decisions

A. Towards Multi-Tier Distributed Architectures

Future iterations of DRPS aim to support architectures beyond the two-tier (edge–core) model. Real-world systems increasingly adopt multi-level hierarchies such as:

- Device → Micro-edge → Regional edge → Cloud
- ISP-level edges and telecom MEC nodes
- CDN hierarchical structures

Introducing multi-tier modelling would allow DRPS to represent a wider variety of modern platforms, especially systems in 5G MEC networks and global content delivery systems.

B. Integration with Real-Time Telemetry

The roadmap considers the ability to incorporate live metrics from running systems. This would allow DRPS to:

- dynamically adjust replica placement based on observed latency spikes
- automatically tune replication factor under fluctuating workloads
- predict congestion based on historical logs

Such hybrid analytical-telemetric systems would make DRPS relevant for operational decision-making, not just educational use.

C. Machine Learning–Enhanced Prediction Models

While the current analytical model is deterministic for clarity, integrating ML in future versions can enable:

- replica popularity forecasting
- adaptive cost modelling based on real-world provider pricing
- optimal configuration prediction using reinforcement learning

This would retain transparency but allow more nuanced behaviours similar to modern adaptive systems.

D. Support for Consistency Model Selection

A future extension includes enabling students to explore how replica placement interacts with consistency guarantees:

- strong consistency vs. eventual consistency
- quorum-based models
- read/write consistency trade-offs

This would bridge replication planning with CAP theorem considerations, improving holistic understanding.

E. 3D Environment Visualization

3D visualization of nodes, distances, and cost–latency heat maps can significantly enhance comprehension. Students often grasp geometric relationships faster when presented spatially, particularly in distributed systems where physical locality matters

REFERENCES

[1] Ali, B., Gregory, M. A., & Li, S. (2021). "Multi-access edge computing architecture, data security and privacy: A review," *IEEE Access*, vol. 9, pp. 18706–18721, doi: 10.1109/ACCESS.2021.3052576.

[2] Amazon Web Services. (2024). *Data Replication and Availability Models*. AWS Whitepaper. Retrieved from https://aws.amazon.com/whitepapers/

[3] Brewer, E. (2012). "CAP twelve years later: How the 'rules' have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23–29, doi: 10.1109/MC.2012.37.

[4] Chiang, M. L., Hsieh, H. C., Chang, T. Y., Lin, T. L., & Chen, H. W. (2023). "An adaptive replica configuration mechanism based on predictive file popularity in mobile edge computing," *Soft Computing*, vol. 27, no. 1, pp. 107–129, doi: 10.1007/s00500-022-07409-y.

[5] Corbett, J. C., Dean, J., Epstein, E., et al. (2013). "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 8:1–8:22, doi: 10.1145/2491245.

[6] DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). "Dynamo: Amazon's highly available key-value store," *Proceedings of 21st ACM Symposium on Operating Systems Principles (SOSP)*, pp. 205–220, doi: 10.1145/1294261.1294281.

[7] Edwin, E. B., Umamaheswari, P., & Thanka, M. R. (2019). "An efficient and improved multi-objective optimized replication management with dynamic and cost aware strategies in cloud computing data center," *Cluster Computing*, vol. 22, no. 5, pp. 11119–11128, doi: 10.1007/s10586-017-1177-9.

[8] Ferrucci, L., Mordacchini, M., & Dazzi, P. (2024). "Decentralized replica management in latency-bound edge environments for resource usage minimization," *IEEE Access*, vol. 12, pp. 1–15, doi: 10.1109/ACCESS.2024.XXXXXXX.

[9] Li, C., Tang, J., & Luo, Y. (2019). "Scalable replica selection based on node service capability for improving data access performance in edge computing environment," *The Journal of Supercomputing*, vol. 75, no. 11, pp. 7209–7243, doi: 10.1007/s11227-019-02848-y.

[10] Li, C., Zhang, Y., & Luo, Y. (2020). "Adaptive replica creation and selection strategies for latency-aware application in collaborative edge-cloud system," *The Computer Journal*, vol. 63, no. 9, pp. 1338–1354, doi: 10.1093/comjnl/bxz151.

[11] Li, C., Liu, J., Lu, B., & Luo, Y. (2021). "Cost-aware automatic scaling and workload-aware replica management for edge-cloud environment," *Journal of Network and Computer Applications*, vol. 180, pp. 103017, doi: 10.1016/j.jnca.2021.103017.

[12] Spring Boot Documentation. (2024). *Building REST APIs with Spring Boot Framework*. Pivotal Software, Inc. Retrieved from https://spring.io/projects/spring-boot/

[13] Tervakari, A. M., Silius, K., Koro, J., Paukkeri, J., & Pirttilä, O. (2016). "Interactive visualization tools to improve learning and teaching in online learning environments," *International Journal of Distance Education Technologies*, vol. 14, no. 1, pp. 1–21, doi: 10.4018/IJDET.2016010101.

[14] Zhang, T. (2023). "Optimizing data replication in edge computing environments," *IEEE Access*, vol. 11, pp. 50123–50135, doi: 10.1109/ACCESS.2023.3278145.

APPENDIX: Deployment Details

Platform: Render Cloud
Backend: Java 17, Spring Boot 3.0
Frontend: HTML5, CSS3, JavaScript
Visualization: Chart.js 4.0
Tested Browsers: Chrome, Edge
URL: https://drps-algorithm.onrender.com